Subset Sum as a Stress Test for Certified Coverage

Jonathan R. Landers

October 2026

Companion note II of III

Abstract. The first note gives a costed model of certified coverage in a 2^n-state search space. Here subset sum supplies a concrete test: exact half-sum summaries, sound interval and residue exclusions, and learned proposals can all be implemented. On 18 constructed no-solution cases that pass cheap root checks, the tested paired solver is slower than meet-in-the-middle in every case. A probability classifier ranks held-out instances well, but its most confident NO proposals still contain YES cases. The experiments identify where this implementation's costs lie; they do not establish a new exact algorithm. The third note asks a different, probability-valued question about reachable targets.

A concrete search space

For positive integers $w_1, \ldots, w_n$ and target t, define

$$Y(w,t) = \mathbf{1}\left\{\exists x \in \{0,1\}^n : \sum_{i=1}^{n} x_i w_i = t\right\}.$$

The 2^n vectors x are the direct-search states. If the weights are split into groups A and B, let

$$\Sigma(A) = \left\{\sum_{i\in A} x_i w_i : x_i \in \{0,1\}\right\}, \qquad \Sigma(A \cup B) = \Sigma(A) + \Sigma(B),$$

where $+$ denotes the set of pairwise sums. Thus $Y(w,t) = 1$ exactly when some $a \in \Sigma(A)$ has $t - a \in \Sigma(B)$. Distinct subsets with the same sum share one exact summary value.

The tested paired search stores distinct sums for a sorted half of the weights and explores the other half with exact interval and residue checks. Meet-in-the-middle (MITM) constructs both half-sum sets and checks complements directly. A bounded depth-first search (DFS) provides a simpler pruning baseline. The algorithms answer the same decision question; the issue is whether the extra pairing and certificates cost less than the work they avoid.

Experiments and controls

An initial set of five constructed families compared bitset dynamic programming, MITM, bounded DFS, and the paired search at several sizes. All applicable exact implementations agreed with exhaustive search on 120 small cases. Narrow weights favored bitset dynamic programming. A mixed planted-solution family made pairing look fast against MITM, but bounded DFS was faster still. A modular no-solution family was rejected at the root by a residue-10 check; that simple arithmetic certificate could also be given to a stronger baseline. The complete timings are in the repository's `subset-sum/RESULTS.md`.

The follow-up deliberately removes that easy explanation. We generated no-solution targets near half the total of random 41-bit odd weights, with each distinct value occurring once, twice, or three times. Sizes were $n \in \{24, 30, 36\}$, with two seeds for each family-size combination. Every target passed whole-instance bounds and attainable-residue tests modulo $8, 10, 16, 32$, and every list had gcd 1. The same cheap root checks were charged to all algorithms. Small validation cases agreed with exhaustive search.

Table 1: Median paired-search time divided by MITM time across two seeds. Values above 1 favor MITM.

Weight multiplicity	$n = 24$	$n = 30$	$n = 36$
Unique	5.16	4.96	4.78
Pairs	4.33	4.26	5.27
Triples	6.29	5.48	5.89

The paired search won 0 of 18 individual comparisons. For example, at $n = 36$, median MITM and paired times were 148 and 707 ms for unique weights, 14.6 and 77.1 ms for pairs, and 3.6 and 21.1 ms for triples. Across all cases the paired search made 0 residue-based branch rejections; interval bounds rejected 1,424 of 1,382,730 visited branch states. MITM already compresses duplicate half sums, leaving little added work for these certificates to save.

These are Python timings on small synthetic families, not a worst-case comparison with optimized subset-sum solvers. The raw measurements and generation rules are in `subset-sum/HARD_RESULTS.md` and `hard_experiment.py`.

Learned proposals

To test the learned-proposal part of the first note, a random forest was trained to predict $Y(w, t)$ from summary features of a weight list and target. Labels were computed exactly by MITM. The data comprised independent-weight and repeated-pair families at $n = 16, 18, 20, 22$; four targets were drawn per list from the middle 40% of its total sum, conditional on passing the cheap modular checks. Training, calibration, and test used disjoint lists: 840, 180, and 180, respectively. Another 240 lists at $n = 24$ tested an unseen size.

On the 720 held-out targets, the forest had ROC AUC 0.866, Brier score 0.133, and accuracy 80.8%. A simple density-based logistic model had AUC 0.864 and Brier score 0.136. The forest ranked cases, but it did not reveal a clear advantage over density. Of 43 targets assigned $P(\text{YES}) \leq 1\%$, 2 were actually YES.

Table 2: Held-out NO proposals at several thresholds. A wrong YES would make uncertified elimination incorrect.

Propose NO if $\widehat{p} \leq$	Proposed NO	Actually YES
1%	43	2
5%	193	7
10%	303	19
20%	406	39
50%	509	78

Interpretation

The first note's coverage criterion counts only soundly eliminated or exactly represented states and charges for finding those representations. Here exact half-sum representation is easy to state but expensive to construct. The tested residue certificates were either decisive at the root on an easy family or nearly inactive after easy cases were excluded. Learned scores can order exact checks; they cannot delete a predicted NO without a sound verifier.

This negative result is useful scope information. It motivates the third note's narrower objective: for a specified random target, bound the *fraction of targets* that are reachable. That probability question admits exact upper and lower bounds even when one target's decision answer remains binary.

Series and reproducibility. Note I: *Certified Coverage of Exponential Search Spaces*. Note III: *Probability-Valued Subset Sum*. Source, figures, and raw data: `https://github.com/jonland82/exponential-search-spaces`.